

[BOJ] 2798(C++/cpp)

☒ 미완	☒
☐ 언어	C++
☐ 문제 출제	BOJ
☐ 진행 상황	성공
☐ 알고리즘 분류	브루트포스
☒ 복습 필요	☐
☐ 풀이날	@2025년 3월 8일
☐ 난도	브론즈2
☒ 발전 가능성 여부	☐

문제

<https://www.acmicpc.net/problem/2798>

백준 2798번 - 블랙잭

N장의 카드 중에서 3장을 골라 그 합이 M을 넘지 않으면서 최대한 가까운 값을 찾는 문제입니다.

풀이 1. 브루트포스를 이용한 탐색

아이디어

- 삼중 반복문을 사용하여 모든 경우의 수를 탐색
- 3개의 카드 조합을 만들어 합을 계산하고, M 이하인 최대값을 찾음

코드

```
#include <iostream>
#include <vector>

using namespace std;
```

```

int main() {
    int n, m;
    cin >> n >> m;

    vector<int> nums(n);

    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    int sum = 0, ans = 0;

    for (int i = 0; i < n - 2; i++) {
        for (int j = i + 1; j < n - 1; j++) {
            for (int k = j + 1; k < n; k++) {
                sum = nums[i] + nums[j] + nums[k];

                if (sum <= m && ans < sum) {
                    ans = sum;
                }

                if (ans == m) {
                    break;
                }
            }
        }
    }

    cout << ans;

    return 0;
}

```

해결 과정에서 어려웠던 점

1. 삼중 반복문을 사용했을 때 시간 복잡도에 대한 고민

- $O(N^3)$ 의 시간 복잡도를 가지므로 N 이 클 경우 비효율적일 수 있음

- 하지만 문제에서 $N \leq 100$ 이므로 최악의 경우 161700번 연산으로 충분히 가능

2. 불필요한 탐색을 최소화하는 방법

- `if (ans == m)` 조건을 통해 더 이상 탐색할 필요가 없으면 중단

풀이 2. 투 포인터를 활용한 최적화

아이디어

- 배열을 정렬한 후, 첫 번째 카드를 고정하고 나머지 두 카드를 투 포인터로 조정
- `left` 는 두 번째 카드, `right` 는 세 번째 카드
- `sum` 을 계산한 후 다음과 같이 조건을 조정
 - `sum == m` 이면 정답을 찾았으므로 즉시 반환
 - `sum < m` 이면 `left++` (더 큰 값으로 이동)
 - `sum > m` 이면 `right--` (더 작은 값으로 이동)
- 시간 복잡도: $O(N^2)$ 로 개선됨

코드

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<int> cards(n);

    for (int i = 0; i < n; i++) {
        cin >> cards[i];
    }

    // 투 포인터를 위한 정렬
    sort(cards.begin(), cards.end());
```

```

int cnt = 0;
int ans = 0;

// i는 첫 번째 카드의 인덱스를 의미
// left가 두 번째, right가 세 번째 카드
for (int i = 0; i < n - 2; i++) {
    int left = i + 1;
    int right = n - 1;

    while (left < right) {
        int sum = cards[i] + cards[left] + cards[right];

        // m과 일치하는 합을 찾아내면 바로 정리
        if (sum == m) {
            std::cout << sum;
            return 0;
        }

        // 합이 작은 경우
        if (sum < m) {
            ans = max(ans, sum);
            left++;
        }
        else {
            right--;
        }
    }
}

std::cout << ans;

return 0;
}

```

해결 과정에서 어려웠던 점

1. 투 포인터의 탐색 방식에 대한 이해

- 기존의 삼중 반복문에서 투 포인터를 활용하려면, **정렬이 필수적으로 필요함**
- `left++` 과 `right--` 을 올바르게 조정해야 최적의 결과를 얻을 수 있음
- i 인덱스를 left와 right의 가운데에 두 번째 카드로 두었다가, 반복문을 활용하는 방식이 애매해져 오래 고민했었음
→ i 인덱스를 1번 카드로 두고, left와 right의 해당하는 카드를 2번, 3번 카드로 두는 방식으로 해결

2. 투 포인터의 종료 조건

- `while (left < right)` 이므로, `left == right` 가 되면 탐색 종료